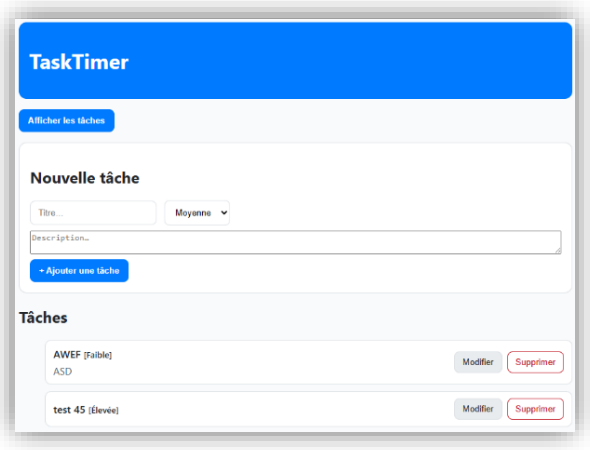
Module : **450** – **Tester des applications**

Table des matières

1 Contexte 2
2 Objectifs du projet 2
3 Description générale du projet 3
4 Organisation du travail 4
5 Travaux à réaliser 4
5.1 **Étape 1** – Mise en place de l'environnement de test 4
5.2 **Étape 2** – Analyse de l'application et identification des anomalies 5
5.3 **Étape 3** – Élaboration du concept de test 6
5.4 **Étape 4** – Élaboration de la stratégie de test et du plan de test 7
6 Critères d'évaluation 9

1 Contexte

Le projet **TaskTimer** simule une mission de test pour l'entreprise **DevFactory**. Vous jouez le rôle d'ingénieur QA (Quality Assurance) chargé d'évaluer la qualité d'une application **Adonis.js existante**, de **planifier**, **concevoir**, **exécuter** et **automatiser des tests**, puis **d'analyser les anomalies** et **proposer** des **mesures correctives**.



L'application TaskTimer est volontairement simple sur le plan fonctionnel, afin que l'accent soit mis sur :

- L'analyse **de la qualité logicielle**
- La **définition d'un concept de test**
- La **mise en place d'une stratégie de test**
- L'**identification et la qualification des anomalies**

Le projet suit une démarche proche de la réalité professionnelle :

Comprendre avant de tester, tester avant de corriger.

L'application livrée contient plusieurs erreurs intentionnelles que vous devez identifier et documenter.

2 Objectifs du projet

- **Analyser** une application existante (backend + frontend)
- **Comprendre** le périmètre fonctionnel d'un projet
- **Définir** un **concept de test**
- **Proposer** une **stratégie de test adaptée**
- **Identifier**, décrire et classifier des anomalies
- **Justifier** des choix de tests (priorité, gravité, risques)
- **Utiliser** des outils de test (API et E2E)

3 Description générale du projet

TaskTimer est une application web de gestion de tâches.

Architecture technique

Backend

- Framework : AdonisJS v6
- Langage : TypeScript
- Base de données : MySQL
- Rôle : exposer une API REST pour gérer les tâches

Frontend

- Framework : **Vue 3**
- Outil de build : **Vite**
- Communication avec le backend via /api
- Rôle : interface utilisateur pour créer, afficher, modifier et supprimer des tâches

Comportement fonctionnel attendu

- L'application **affiche la liste** des tâches.
- L'utilisateur peut **créer** une tâche
  (Titre obligatoire, Priorité = Faible/Moyenne/Élevée, Description optionnelle).
  Liste mis à jour après ajout.
- L'utilisateur peut **modifier** une tâche existante (Titre/Priorité/Description).
  Liste mis à jour après modification.
- L'utilisateur peut **supprimer** une tâche.
  Liste mis à jour après modification après suppression.
- Les messages d'erreur sont affichés de manière **claire et visible**.

Contraintes d'interface (UI & accessibilité)

La charte UI suivante représente la référence visuelle imposée par le client DevFactory. Toute divergence par rapport à cette charte doit être identifiée, classée (gravité/priorité) et documentée dans le rapport.:

- **Libellés FR** et cohérents (ex. bouton **"+ Ajouter une tâche"**).
- **Couleurs** : actions principales en **bleu #007BFF**, erreurs **texte #c1121f** sur **fond #ffeef0**.
- **Priorité** : afficher **Faible/Moyenne/Élevée**
- **Lisibilité** : police sans-serif, contrastes suffisants, messages d'erreur proches de l'action en cause.
- Messages d'erreur : **rouges**, format "Erreur : [message]"
  doivent apparaître directement sous le bouton d'action concerné
- **Comportement UI** : pas d'ajout "optimiste" silencieux ; l'UI reflète l'état réel de l'API (liste mise à jour seulement après succès).

4 Organisation du travail

- Travail individuel ou en binôme (selon consigne)
- Versionnage recommandé (Git)
- Utilisation d'un journal de travail
- Rendus structurés et clairs

5 Travaux à réaliser

Le projet **TaskTimer** se déroule sur environ 24 périodes et est structuré en plusieurs étapes successives.

Chaque étape correspond à un livrable distinct, évalué séparément.

Les tests automatisés ne seront développés qu'à partir de l'étape 4.

5.1 Étape 1 – Mise en place de l'environnement de test

Document à rendre : Rapport de mise en place
→ **S-P_450-E1_env_test-grp_xxx_yyy** avec xxx_yyy vos noms ou prénoms

Objectif

Installer et exécuter l'application **TaskTimer** dans un environnement local afin de disposer d'une base fonctionnelle pour les activités de test.

Travaux demandés

Vous devez :

- Installer l'application backend (AdonisJS)
- Installer l'application frontend (Vue + Vite)
- Configurer l'environnement (base de données, variables d'environnement)
- Vérifier que l'application :
  - Démarre correctement
  - Affiche l'interface utilisateur
  - Permet d'interagir avec les fonctionnalités principales

Attendus

- L'application doit être **exécutable localement**
- Les éventuels problèmes d'installation doivent être **documentés**
- Aucun correctif fonctionnel n'est attendu à ce stade
  Toute correction de bug ou modification fonctionnelle effectuée à cette étape ne sera pas prise en compte.

Cette étape valide la capacité à préparer un environnement de test stable.

5.2 Étape 2 – Analyse de l'application et identification des anomalies

Document à rendre : Analyse des anomalies
→ **S-P_450-E2_anomalies-grp_xxx_yyy** avec xxx_yyy vos noms ou prénoms

Objectif

Analyser l'application fournie afin d'identifier les **défauts, bugs et anomalies**, sans encore écrire de tests automatisés.

Travaux demandés

Vous devez :

- Explorer l'application (UI + API)
- Identifier les anomalies :
  - Fonctionnelles
  - Techniques
  - Ergonomiques (UI / UX)
- Décrire chaque anomalie de manière claire et reproductible
- Classifier chaque anomalie selon :
  - **Gravité**
  - **Sévérité**
  - **Priorité**

Format attendu

Un tableau d'analyse des anomalies comprenant au minimum :

- Identifiant de l'anomalie (B-01, B-02, …)
- Description
- Étapes de reproduction
- Résultat attendu
- Résultat observé
- Gravité (Bloquante, Sérieuse, Mineure, Cosmétique)
- Sévérité (Critique, Majeure, Mineure, Triviale)
- Priorité (Haute, Moyenne, Basse)

Aucune correction n'est effectuée à ce stade.
Le but est d'apprendre à **observer, qualifier et prioriser**.

5.3 Étape 3 – Élaboration du concept de test

Document à rendre : Concept de test
→ **S-P_450-E3_concept_test-grp_xxx_yyy** avec xxx_yyy vos noms ou prénoms

Objectif

Définir le **cadre global** des activités de test pour le projet TaskTimer.

Travaux demandés

Vous devez élaborer un **concept de test** décrivant notamment :

- **Objectifs du test** : à quoi sert la campagne de tests sur TaskTimer ?
- **Périmètre du test** : quelles parties du code ou fonctionnalités seront testées (et lesquelles ne le seront pas) ?
- **Types de tests prévus** : ex : unitaires, d'intégration, E2E, de performance, de sécurité, UI.
- **Ressources** : outils, environnements, jeux de données, contraintes techniques.
- **Risques identifiés** : problèmes potentiels, dépendances externes, limites techniques.

Le concept de test répond à la question :

Quelle est notre vision globale du test pour ce projet ?

Contraintes

- Le concept de test doit être **adapté au projet TaskTimer**
- Il doit être cohérent avec les anomalies identifiées à l'étape 2
- Il ne contient **pas encore de cas de test détaillés**

5.4 Étape 4 – Élaboration de la stratégie de test et du plan de test

Document à rendre : Comprenant

- Stratégie de test
- Plan de test
- Tests

→ **S-P_450-E4_strategie_test-grp_xxx_yyy** avec xxx_yyy vos noms ou prénoms

Objectif

Transformer le concept de test en une **démarche opérationnelle**, puis mettre en œuvre les tests.

Travaux demandés

Vous devez :

1. Définir une **stratégie de test** :

La stratégie de test décrit comment les tests seront organisés et menés pour le projet TaskTimer.

- **Approche basée sur les risques**
  Vous devez :
  - Identifier les risques principaux liés à l'application
  - Relier ces risques aux anomalies déjà observées
  - Expliquer l'impact potentiel de chaque risque sur :
    - Le fonctionnement de l'application
    - L'utilisateur
    - Les données

  L'approche doit montrer que tous les tests n'ont pas la même importance

- **Priorisation des tests**
  Vous devez :
  - Définir quels tests sont prioritaires
  - Justifier cette priorisation en fonction :
    - De la gravité des anomalies
    - De la fréquence d'utilisation des fonctionnalités
    - De l'impact en cas de défaillance

  Les tests doivent être classés (ex. priorité haute, moyenne, basse).

- **Choix des outils**
  Vous devez :
  - Indiquer les outils de test utilisés
  - Expliquer pourquoi chaque outil est choisi
  - Faire le lien entre :
    - Le type de test
    - L'outil utilisé

  Le choix doit être justifié, pas seulement listé.

- **Ordre d'exécution des tests**
  Vous devez :
  - Définir l'ordre logique dans lequel les tests seront exécutés
  - Expliquer pourquoi cet ordre est pertinent
  - Faire le lien avec la **pyramide de tests**

2. Élaborer un **plan de test** :

Le plan de test décrit quoi tester concrètement

- **Scénarios de test**

Vous devez :

- Définir des scénarios de test
- Décrire des situations représentatives de l'utilisation réelle de l'application
- Couvrir :
  - Les cas normaux
  - Les cas d'erreur
  - Les cas limites

Un scénario décrit un enchaînement logique d'actions.

- **Cas de test**

Vous devez :

- Détailler un ou plusieurs cas de test
- Préciser :
  - Les données d'entrée
  - L'action effectuée
  - Le résultat attendu

Un cas de test doit être clair, précis et reproductible.

- **Conditions de réussite**

Vous devez :

- Définir les critères permettant de dire qu'un test est réussi ou échoué
- Préciser :
  - Le résultat attendu
  - Le comportement acceptable / non acceptable
  - Éviter les formulations vagues ("ça fonctionne")

Les conditions de réussite doivent être objectives et mesurables.

3. **Développer les tests automatisés** :

À partir du plan de test, les élèves implémentent les tests automatisés.

- **Tests unitaires**

Vous devez :

- Tester une logique isolée
- Vérifier des règles de validation ou de calcul
- Éviter toute dépendance :
  - À la base de données
  - Au réseau
  - À l'interface utilisateur

Les tests unitaires doivent être rapides et ciblés.

- **Tests d'intégration (API)**

Vous devez :

- Tester les endpoints de l'API REST
- Vérifier :
  - Les codes HTTP
  - Les réponses JSON
  - L'intégration avec la base de données
- Couvrir :
  - Les cas valides
  - Les cas invalides
  - Les erreurs attendues

Ces tests vérifient que plusieurs composants fonctionnent ensemble.

- **Tests End-to-End (E2E)**

Les élèves doivent :

- Tester des parcours utilisateur complets via l'interface
- Simuler les actions réelles d'un utilisateur
- Vérifier :
  - L'affichage
  - Les messages d'erreur
  - Le résultat final du scénario

Les tests E2E valident que l'application fonctionne du point de vue de l'utilisateur.

Le développement des tests est limité à cette étape uniquement.

Consignes pour l'étape 4

Dans le cadre de l'étape 4, vous devez démontrer que les tests automatisés ont été exécutés, indépendamment de la méthode utilisée pour les écrire (TDD ou non).

Le TDD n'est pas imposé.

Cependant, vous devez être capables de prouver l'exécution effective des tests.

Preuves attendues (obligatoires)

Pour chaque type de test mis en place, vous devez fournir **au minimum une copie d'écran** montrant :

- La commande d'exécution des tests
- Le résultat de l'exécution
- L'état final des tests (succès / échec)

6 Critères d'évaluation

Une grille d'évaluation détaillée est mise à disposition au format PDF (voir consignes enseignant).

Cette grille présente l'ensemble des critères d'évaluation utilisés pour le projet TaskTimer, en lien direct avec les différentes étapes du travail demandé.

Elle permet de :

- Comprendre ce qui est attendu à chaque étape,
- Identifier les éléments évalués (analyse, concept de test, stratégie, tests),
- Clarifier le niveau de qualité attendu (Acquis / Partiellement acquis / Non acquis).

Vous êtes fortement encouragés à vous référer à cette grille tout au long du projet afin d'autoévaluer votre travail et de vérifier la cohérence entre vos documents, vos démarches de test et les livrables attendus.